This work is licensed under a Creative Commons Attribution 4.0 International License .

Trabalho Prático

DevMatch: Especificação e Modelagem de uma Plataforma de Marketplace para Contratação de Profissionais de TI Freelancers

DevMatch: Specification and Modeling of a Marketplace Platform for Hiring Freelance IT Professionals

Antônio Henrique Rabelo Andrade
[PUC Minas | *ahrandrade@sga.pucminas.br*]
João Pedro Bomfim Victor
[PUC Minas | *joao.bomfim@sga.pucminas.br*]
Júlia de Souza Ventura
[PUC Minas | *jsventura@sga.pucminas.br*]
Pedro Henrique Germano de Melo Santos
[PUC Minas | *pedro.germano@sga.pucminas.br*]
Yuri Cardoso Viana
[PUC Minas | *ycviana@sga.pucminas.br*]

Curso de Engenharia de Software, Pontifícia Universidade Católica de Minas Gerais (PUC Minas), Belo Horizonte, MG, Brasil.

Resumo. Este artigo especifica o DevMatch, marketplace de contratação de profissionais de TI freelancers. A proposta integra publicação de projetos, recomendação, contratos por marcos, custódia de pagamentos e mediação de disputas. Com base no enunciado, foram descritos quatro atores, casos de uso e histórias de usuário com critérios de aceitação. O resultado é uma especificação inicial do fluxo de contratação, que orienta a modelagem de projeto e de dados. Implementação e validação com usuários são etapas futuras.

Abstract. This article specifies DevMatch, a marketplace for hiring freelance IT professionals. The proposal combines project posting, recommendations, milestone contracts, escrow payments, and dispute mediation. Based on the system brief, four actors, use cases, and user stories with acceptance criteria were described. The result is an initial specification of the hiring workflow that guides the design and data models. Implementation and validation with users remain future work.

Palavras-chave: Marketplace de Freelancers, Engenharia de Software, Modelagem de Sistemas, UML

Keywords: Freelancer Marketplace, Software Engineering, System Modeling, UML

1 Introdução

A expansão das plataformas digitais modificou a forma como empresas e profissionais estabelecem relações de trabalho. No setor de tecnologia da informação, essas plataformas possibilitam que organizações e pessoas contratem profissionais independentes para desenvolver sistemas, aplicativos, páginas Web, integrações, automações e outros produtos de software. Ao mesmo tempo, profissionais freelancers podem encontrar oportunidades sem as limitações geográficas de processos tradicionais de contratação. A existência de mercados de trabalho mediados por plataformas reforça a relevância de estudar esse modelo de contratação [**?**].

Apesar de suas vantagens, a contratação remota de serviços de desenvolvimento de software envolve riscos para clientes e freelancers. Os clientes podem encontrar dificuldades para avaliar as competências técnicas dos candidatos, verificar seus históricos profissionais e acompanhar a execução do trabalho contratado. Por outro lado, freelancers estão sujeitos a alterações de escopo, requisitos pouco claros, atrasos na aprovação das entregas e falta de pagamento. Em ambientes digitais, mecanismos de reputação, avaliação e rastreabilidade são importantes para reduzir incertezas e apoiar a construção de confiança entre participantes que não possuem uma relação anterior [**?**].

Nesse contexto, propõe-se o DevMatch, uma plataforma de marketplace destinada a conectar profissionais de tecnologia freelancers a empresas ou pessoas físicas que necessitam contratar serviços de desenvolvimento de software. O sistema permite que clientes publiquem projetos contendo escopo, prazo, orçamento estimado e tecnologias exigidas. Os freelancers, por sua vez, podem cadastrar suas competências, portfólio, certificações, disponibilidade e valor de trabalho, além de localizar projetos e enviar propostas comerciais.

A proposta do DevMatch integra mecanismos voltados à confiança e à segurança da contratação. Entre eles estão a recomendação inteligente de projetos e profissionais, a verificação de competências por meio de testes técnicos, a avaliação bidirecional, o acompanhamento por marcos de entrega e o pagamento em custódia. Nesse modelo, o valor de cada marco permanece em custódia até a aprovação da entrega ou a resolução de uma disputa, segundo as regras contratuais. O sistema também permite a abertura de disputas, a mediação por administradores e a manutenção de registros auditáveis das alterações contratuais e transações financeiras.

O objetivo geral deste trabalho é especificar e modelar o sistema DevMatch, aplicando conceitos de engenharia de requisitos, modelagem UML, arquitetura de software e modelagem de dados. Como objetivos específicos, busca-se iden-

tificar os atores e suas responsabilidades; levantar requisitos funcionais e não funcionais; representar as interações por meio de casos de uso e histórias de usuário; definir a arquitetura do sistema; elaborar diagramas estruturais e comportamentais; e desenvolver os modelos conceitual, lógico e físico do banco de dados. O processo de levantamento e documentação dos requisitos segue princípios de definição, análise e especificação de necessidades estabelecidos pela engenharia de requisitos [**?**].

Este artigo está organizado da seguinte maneira: após esta introdução, são apresentados os trabalhos relacionados e as características que diferenciam o DevMatch de soluções semelhantes. Em seguida, a seção de modelagem de requisitos descreve os atores, os casos de uso, as histórias de usuário, os diagramas de sequência do sistema e os contratos de operação. Posteriormente, são apresentadas a arquitetura e as modelagens de projeto e de dados. Por fim, são discutidos os resultados da modelagem e apresentadas as considerações finais do trabalho.

2 Trabalhos Relacionados

3 Modelagem de Requisitos

A modelagem de requisitos do DevMatch tem como finalidade definir as responsabilidades dos participantes e delimitar as funcionalidades pertencentes ao escopo da plataforma. Os requisitos foram obtidos a partir da análise do contexto do sistema, considerando o ciclo completo de contratação: publicação do projeto, busca e recomendação de profissionais, envio de propostas, formalização do contrato, acompanhamento das entregas, pagamento, disputa e avaliação.

Os comportamentos identificados foram representados por meio de casos de uso, seguindo a notação da Unified Modeling Language (UML) [**?**]. As necessidades dos participantes também foram documentadas como histórias de usuário, permitindo relacionar cada funcionalidade ao benefício esperado por seu respectivo ator.

3.1 Descrição dos Atores

Um ator representa um papel externo que interage com o sistema para atingir determinado objetivo. No DevMatch foram identificados quatro atores principais: freelancer, cliente, administrador do sistema e analista financeiro ou de suporte. A Tabela 1 apresenta os objetivos e as principais interações de cada um deles.

O freelancer e o cliente são os atores diretamente envolvidos na contratação e execução de um projeto. O administrador e o analista financeiro ou de suporte são atores internos da organização responsável pelo DevMatch. Embora possuam acesso a informações operacionais, suas permissões devem ser limitadas conforme suas responsabilidades, respeitando os requisitos de privacidade, segurança e rastreabilidade.

Serviços externos, como processadores de pagamento, mecanismos de autenticação e serviços de notificação, podem participar da execução técnica das funcionalidades. Entretanto, eles são considerados sistemas de apoio e não substituem os quatro atores de negócio identificados.

3.2 Modelo de Casos de Uso

O modelo de casos de uso apresenta uma visão geral das funcionalidades oferecidas pelo DevMatch e das relações entre essas funcionalidades e seus atores principais. O diagrama da Figura 1 apresenta esses objetivos de negócio; o catálogo complementa as interações compartilhadas.

O caso de uso de gerenciamento de perfil inclui o registro de competências, portfólio, certificações, disponibilidade e valor de trabalho do freelancer. A comprovação de competências ocorre por meio de testes técnicos associados a tecnologias específicas, com resultado registrado no perfil. O mecanismo de recomendação considera stack, disponibilidade, histórico de avaliações e faixa de preço para sugerir projetos a freelancers e profissionais a clientes. As sugestões auxiliam a escolha, sem realizar contratações automaticamente.

Após a publicação de um projeto, os freelancers podem enviar propostas contendo valor, prazo e observações. Quando uma proposta é aceita, cliente e freelancer formalizam um contrato e definem os marcos de entrega. O cliente realiza o depósito do valor acordado no sistema de custódia, e o pagamento é liberado de acordo com a aprovação das entregas. Caso exista desacordo, uma disputa pode ser aberta e encaminhada para mediação.

A Tabela 2 apresenta o catálogo dos principais casos de uso identificados.

3.2.1 Especificação dos Casos de Uso Centrais

UC03 – Enviar e negociar proposta. O ator principal é o freelancer. Como pré-condições, o freelancer deve estar autenticado, possuir perfil ativo e o projeto deve estar aberto para propostas. O fluxo inicia-se quando o freelancer seleciona um projeto, informa valor, prazo e observações e confirma o envio. O sistema valida os dados, registra a proposta e notifica o cliente. Durante a negociação, as partes podem sugerir alterações, mantendo o histórico das versões. Como pós-condição, a proposta permanece registrada como enviada, em negociação, aceita, recusada ou cancelada. O sistema não deve aceitar propostas para projetos encerrados.

UC04 – Publicar e gerenciar projeto. O ator principal é o cliente. O cliente autenticado informa título, descrição do escopo, orçamento estimado, prazo, tecnologias necessárias e critérios de aceitação. O sistema valida os campos obrigatórios e publica o projeto. Após a publicação, o cliente pode editar informações que não alterem propostas já negociadas, pausar o recebimento de propostas ou encerrar o projeto. Como pós-condição, o projeto fica disponível para pesquisa e para o mecanismo de recomendação.

UC06 – Formalizar e acompanhar contrato. O caso de uso é iniciado quando o cliente aceita uma proposta. O sistema utiliza as informações do projeto e da proposta para gerar o contrato, incluindo escopo, valor, prazo, marcos e responsabilidades. Cliente e freelancer devem confirmar os termos. Após as confirmações e a validação do depósito em custódia, o contrato passa para o estado ativo. Alterações posteriores devem ser registradas e aceitas pelas duas partes.

UC07 – Gerenciar marcos e entregas. O freelancer registra uma entrega associada a um marco do contrato e anexa os artefatos ou links necessários. O sistema notifica o cliente, que

Tabela 1. Atores e suas interações com o DevMatch.

Ator	Objetivo	Interações com o sistema
Freelancer	Encontrar oportunidades e prestar serviços de tecnologia de forma segura.	Cria e atualiza seu perfil profissional; informa competências, portfólio, certificações, disponibilidade e valor; realiza testes técnicos; pesquisa projetos; recebe recomendações; envia e negocia propostas; acompanha contratos e marcos; registra entregas; recebe pagamentos; abre disputas; e avalia clientes.
Cliente	Contratar profissionais adequados e acompanhar a execução dos projetos.	Cadastra projetos com escopo, prazo, orçamento e tecnologias; recebe recomendações de freelancers; analisa e negocia propostas; contrata profissionais; deposita valores em custódia; acompanha o progresso; aprova ou rejeita entregas; libera pagamentos; abre disputas; e avalia freelancers.
Administrador	Garantir a segurança, a integridade e o funcionamento adequado da plataforma.	Gerencia usuários, perfis e permissões; acompanha verificações de identidade; bloqueia contas quando necessário; consulta logs e registros de auditoria; analisa comportamentos suspeitos; e realiza a mediação de disputas entre clientes e freelancers.
Analista financeiro/suporte	Administrar operações financeiras e atender solicitações relacionadas ao uso da plataforma.	Acompanha depósitos em custódia; valida repasses e reembolsos; consulta transações; emite relatórios financeiros e de comissões; investiga falhas de pagamento; e atende chamados de clientes e freelancers.

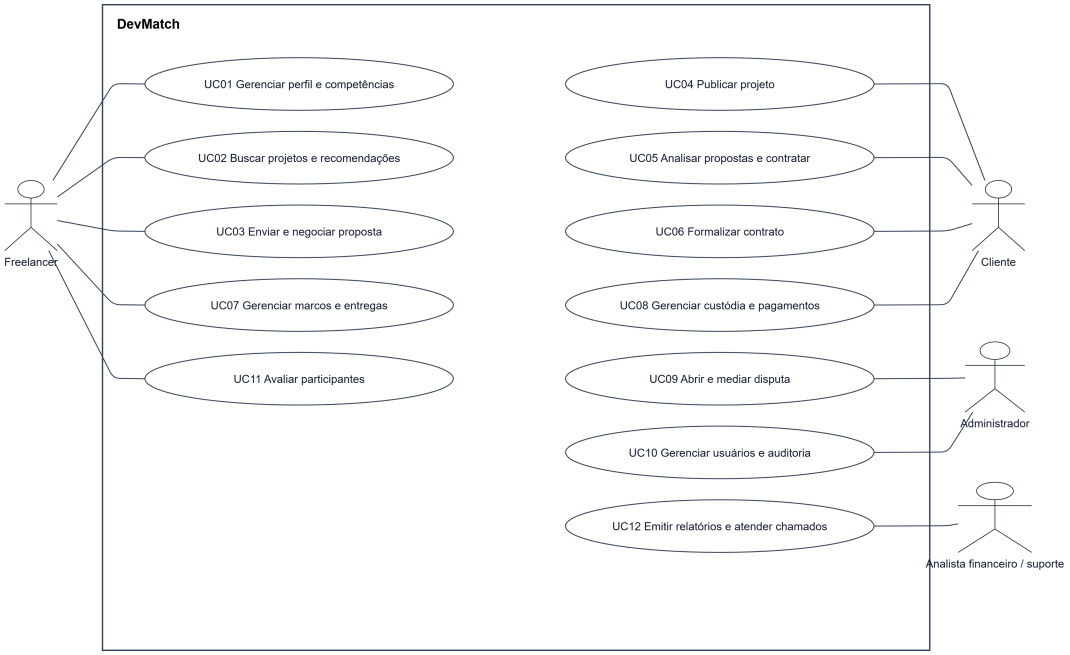


Figura 1. Visão geral dos casos de uso do DevMatch. As associações destacam o ator principal; os demais participantes são descritos no catálogo.

pode aprovar a entrega ou solicitar ajustes com uma justificativa. Quando aprovada, a entrega é registrada como concluída e o processo de liberação do pagamento correspondente é iniciado. Caso o prazo de avaliação expire, o sistema deve aplicar a regra definida no contrato.

UC08 – Gerenciar custódia, pagamentos e repasses. O cliente provisiona em custódia o valor dos marcos conforme as condições do contrato, antes da execução correspondente. O sistema registra a transação e mantém o valor bloqueado até a aprovação do marco ou a resolução de uma disputa. Após uma aprovação, o valor correspondente é liberado para repasse ao freelancer, descontadas as taxas aplicáveis. Falhas, cancelamentos, estornos e reembolsos devem permanecer registrados para fins de auditoria.

UC09 – Abrir e mediar disputa. Cliente ou freelancer pode abrir uma disputa quando houver desacordo relacionado ao escopo, prazo, qualidade da entrega ou pagamento. O solicitante informa a justificativa e apresenta evidências. A outra parte é notificada e pode responder. O administrador analisa o contrato, as mensagens, as entregas e as transações antes de registrar a decisão. Como resultado, o pagamento pode ser liberado, devolvido integralmente ou dividido conforme a decisão da mediação.

3.3 Histórias de Usuário

As histórias de usuário descrevem funcionalidades a partir da perspectiva dos participantes. Cada história adota a es-

Tabela 2. Catálogo dos principais casos de uso do DevMatch.

Código	Caso de uso	Ator principal	Objetivo
UC01	Gerenciar perfil e competências	Freelancer	Cadastrar e manter informações profissionais, portfólio, certificações, disponibilidade, valor e resultados de testes técnicos.
UC02	Buscar e consultar recomendações	Freelancer e cliente	Localizar projetos por filtros e visualizar oportunidades sugeridas ao freelancer ou profissionais sugeridos ao cliente.
UC03	Enviar e negociar proposta	Freelancer	Apresentar ao cliente uma proposta contendo valor, prazo e condições, permitindo ajustes durante a negociação.
UC04	Publicar e gerenciar projeto	Cliente	Cadastrar um projeto com escopo, prazo, orçamento, tecnologias e demais informações necessárias à contratação.
UC05	Analisar propostas e contratar freelancer	Cliente	Comparar propostas e perfis para selecionar o profissional responsável pela execução do projeto.
UC06	Formalizar e acompanhar contrato	Cliente e freelancer	Registrar o acordo entre as partes, incluindo escopo, valor, prazo, responsabilidades e situação do contrato.
UC07	Gerenciar marcos e entregas	Cliente e freelancer	Definir marcos, registrar entregas, solicitar ajustes e acompanhar o progresso do projeto.
UC08	Gerenciar custódia, pagamentos e repasses	Cliente e analista financeiro	Registrar depósitos, manter valores em custódia, liberar pagamentos após aprovação ou mediação, executar repasses e tratar reembolsos.
UC09	Abrir e mediar disputa	Cliente, freelancer e administrador	Registrar um desacordo, anexar evidências, acompanhar a análise e receber a decisão da mediação.
UC10	Gerenciar usuários e auditoria	Administrador	Controlar contas, permissões, verificações de identidade, bloqueios, logs e registros de auditoria.
UC11	Avaliar participantes	Cliente e freelancer	Registrar avaliações após o encerramento do contrato e atualizar a reputação dos participantes.
UC12	Emitir relatórios e atender chamados	Analista financeiro/suporte	Consultar transações, gerar relatórios e solucionar chamados relacionados a pagamentos e uso da plataforma.

trutura “Como ator, quero realizar uma ação, para obter determinado benefício”. Os critérios apresentados delimitam as condições mínimas para que a história seja considerada atendida.

HU01 – Perfil profissional. Como freelancer, quero cadastrar minhas competências, experiências, portfólio, certificações e valor de trabalho, para apresentar minhas qualificações aos clientes.

Critérios de aceitação: o sistema deve exigir os campos obrigatórios; permitir atualização das informações; validar links do portfólio; e registrar a data da última alteração.

HU02 – Verificação de competências. Como freelancer, quero realizar testes técnicos relacionados às tecnologias que conheço, para comprovar minhas habilidades e receber selos em meu perfil.

Critérios de aceitação: o teste deve possuir tempo e regras definidos; o resultado deve ser armazenado; e o selo somente deve ser concedido quando a pontuação mínima for atingida.

HU03 – Busca de projetos. Como freelancer, quero filtrar projetos por tecnologia, orçamento, prazo e categoria, para encontrar oportunidades compatíveis com meu perfil.

Critérios de aceitação: os filtros devem poder ser combinados; os resultados devem apresentar apenas projetos abertos; e o freelancer deve conseguir consultar os detalhes antes de enviar uma proposta.

HU04 – Envio de proposta. Como freelancer, quero enviar uma proposta com valor, prazo e observações, para participar da seleção de um projeto.

Critérios de aceitação: o projeto deve estar aberto; valor e prazo devem ser válidos; o envio deve ser confirmado; e o cliente deve ser notificado.

HU05 – Publicação de projeto. Como cliente, quero publicar um projeto informando escopo, orçamento, prazo e tecnologias necessárias, para receber propostas de profissionais qualificados.

Critérios de aceitação: os campos obrigatórios devem ser validados; o prazo deve ser posterior à data de publicação; e o projeto deve ficar disponível para pesquisa após a confirmação.

HU06 – Análise de propostas. Como cliente, quero comparar propostas e perfis de freelancers, para selecionar o profissional mais adequado ao projeto.

Critérios de aceitação: o cliente deve visualizar valor, prazo, reputação, competências e portfólio; somente propostas do próprio projeto podem ser consultadas; e o aceite deve exigir confirmação.

HU07 – Recomendação inteligente. Como cliente, quero receber recomendações de freelancers compatíveis com meu projeto, para reduzir o tempo necessário para encontrar profissionais.

Critérios de aceitação: a recomendação deve considerar tecnologias, disponibilidade, avaliações e faixa de preço; e o cliente deve conseguir acessar o perfil de cada profissional recomendado.

HU08 – Acompanhamento dos marcos. Como freelancer, quero registrar as entregas correspondentes aos marcos do contrato, para demonstrar o progresso do projeto e solicitar

sua aprovação.

Critérios de aceitação: somente participantes do contrato podem acessar os marcos; cada entrega deve conter data e descrição; e o cliente deve ser notificado após o envio.

HU09 – Aprovação da entrega. Como cliente, quero aprovar uma entrega ou solicitar ajustes, para garantir que o resultado esteja de acordo com o escopo contratado.

Critérios de aceitação: a solicitação de ajustes deve conter uma justificativa; a aprovação deve exigir confirmação; e todas as decisões devem permanecer registradas.

HU10 – Pagamento em custódia. Como cliente, quero depositar o valor do contrato em custódia, para assegurar ao freelancer que o pagamento estará disponível após a aprovação das entregas.

Critérios de aceitação: a transação deve gerar um identificador; o valor deve permanecer bloqueado até a aprovação; e cliente e freelancer devem conseguir consultar a situação do pagamento.

HU11 – Abertura de disputa. Como participante de um contrato, quero abrir uma disputa e apresentar evidências, para solicitar mediação quando não houver acordo sobre uma entrega ou pagamento.

Critérios de aceitação: apenas participantes do contrato podem abrir a disputa; a justificativa deve ser obrigatória; a outra parte deve ser notificada; e a decisão do administrador deve ser registrada.

HU12 – Avaliação bidirecional. Como participante de um contrato concluído, quero avaliar a outra parte, para contribuir com a reputação e com a segurança das futuras contratações.

Critérios de aceitação: a avaliação somente pode ocorrer após o encerramento do contrato; cada participante pode avaliar uma única vez; a nota deve respeitar a escala definida; e o sistema deve recalcular a reputação após o registro.

HU13 – Mediação de disputas. Como administrador, quero consultar contratos, mensagens, entregas e transações relacionados a uma disputa, para tomar uma decisão fundamentada e auditável.

Critérios de aceitação: somente administradores autorizados podem acessar o processo; a decisão deve conter justificativa; e todas as consultas e alterações devem ser registradas em log.

HU14 – Controle financeiro. Como analista financeiro, quero acompanhar depósitos, repasses, reembolsos e comissões, para controlar as operações financeiras da plataforma.

Critérios de aceitação: as transações devem poder ser filtradas; valores não podem ser alterados sem autorização; operações manuais devem registrar o responsável; e os relatórios devem apresentar período, situação e tipo da transação.

As histórias de usuário serão utilizadas como referência para a elaboração dos demais modelos do sistema. As entidades, componentes e operações representados nos diagramas posteriores devem manter consistência com os atores, regras e fluxos definidos nesta seção.

3.4 Diagrama de Sequência do Sistema e Contratos de Operação

4 Modelagem de Projeto

A modelagem de projeto traduz os requisitos apresentados na Seção 3 em uma solução técnica. Nesta seção são definidas a arquitetura do sistema, a decomposição em componentes e a distribuição desses componentes nos nós de execução. As decisões adotadas procuram atender aos atributos de qualidade considerados críticos para o DevMatch: segurança das transações financeiras, rastreabilidade das alterações contratuais, disponibilidade da plataforma e facilidade de evolução das regras de negócio.

4.1 Arquitetura do Sistema

O DevMatch adota uma arquitetura em camadas combinada com o padrão arquitetural Model-View-Controller (MVC). A arquitetura em camadas organiza o sistema em níveis de abstração com responsabilidades distintas, nos quais cada camada utiliza apenas os serviços oferecidos pela camada imediatamente inferior [**?**]. O padrão MVC, por sua vez, separa a representação da informação, a lógica de apresentação e o tratamento das requisições do usuário [**?**].

A escolha combinada justifica-se pelas características do sistema. O DevMatch possui regras de negócio relevantes e independentes da interface, tais como a transição de estados de contratos, marcos e disputas, o bloqueio e a liberação de valores em custódia e o cálculo de reputação. Essas regras precisam permanecer isoladas da camada de apresentação para que possam ser testadas, auditadas e reutilizadas por diferentes clientes, como a aplicação Web e futuros aplicativos móveis. Ao mesmo tempo, a interação com o usuário é intensa e orientada a formulários e listagens, cenário no qual o MVC oferece uma separação adequada entre a exibição dos dados e o tratamento das requisições.

A Figura 2 apresenta a visão geral da arquitetura. A camada de apresentação executa no navegador do usuário e comunica-se com a camada de interface por meio de uma API REST sobre HTTPS. A camada de aplicação orquestra os casos de uso, a camada de domínio concentra as regras de negócio e a camada de persistência isola o acesso ao banco de dados. A camada de integração encapsula a comunicação com serviços externos, como o gateway de pagamento e os provedores de notificação e de identidade. Os serviços transversais de segurança, auditoria e registro de log são acessíveis às demais camadas, sem que estas dependam de sua implementação concreta.

A Tabela 3 descreve as responsabilidades de cada camada e os principais elementos nela alocados.

O padrão MVC é aplicado sobre essa estrutura conforme apresentado na Tabela 4. Adota-se a variante em que a camada de apresentação executa no cliente e consome uma API REST, de modo que a *View* é implementada na aplicação Web, o *Controller* corresponde aos controladores da camada de interface e o *Model* é formado pelas camadas de aplicação, domínio e persistência.

A separação entre *Controller* e *Model* é relevante para os requisitos de segurança e auditoria. Operações sensíveis, como a liberação de um valor em custódia ou o registro da decisão de uma disputa, são executadas exclusivamente por

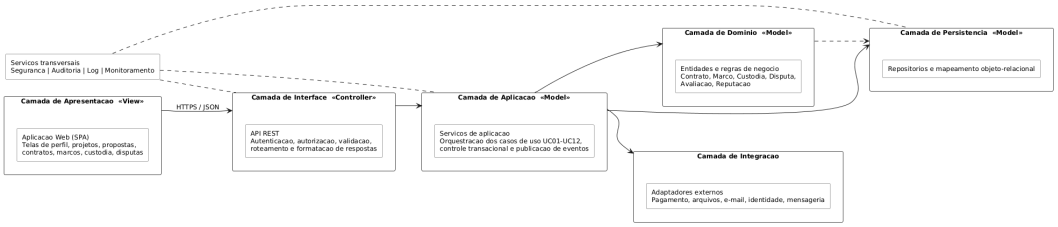


Figura 2. Arquitetura em camadas do DevMatch com a distribuição dos elementos do padrão MVC.

Tabela 3. Camadas da arquitetura do DevMatch.

Camada	Responsabilidade	Elementos principais
Apresentação	Exibir as informações ao usuário, capturar suas ações e validar dados de formulário. Não contém regras de negócio.	Telas de perfil, busca de projetos, propostas, contratos, marcos, custódia, disputas e avaliações; componentes de interface; e roteamento da aplicação Web.
Interface (controle)	Receber as requisições, autenticar e autorizar o solicitante, validar os dados de entrada, delegar a execução à camada de aplicação e formatar a resposta.	Controladores REST (*ProjectController*, *ProposalController*, *ContractController*, *MilestoneController*, *PaymentController*, *DisputeController*); filtros de autenticação; e tratadores de erro.
Aplicação	Orquestrar os casos de uso, coordenar objetos de domínio, controlar transações e publicar eventos de notificação e auditoria.	Serviços de aplicação correspondentes aos casos de uso UC01 a UC12; coordenadores de transação; e mapeadores entre objetos de transferência e entidades.
Domínio	Concentrar as entidades, os objetos de valor e as regras de negócio do sistema, incluindo as máquinas de estado e as invariantes.	Entidades *Usuario*, *Freelancer*, *Cliente*, *Projeto*, *Proposta*, *Contrato*, *Marco*, *Entrega*, *Custodia*, *Transacao*, *Disputa* e *Avaliacao*; e as regras de reputação, de liberação de pagamento e de prazo de aprovação.
Persistência	Isolar o acesso aos dados, traduzindo objetos de domínio em registros do banco de dados e vice-versa.	Repositórios por agregado; mapeamento objeto-relacional; consultas de busca e filtragem; e controle de versão do esquema.
Integração	Encapsular a comunicação com sistemas externos, oferecendo interfaces estáveis ao restante do sistema.	Adaptadores de gateway de pagamento, de armazenamento de arquivos, de envio de mensagens e de provedor de identidade; e tratamento de *webhooks*.
Transversal	Prover funcionalidades utilizadas por várias camadas, sem acoplá-las a implementações específicas.	Controle de acesso baseado em papéis, criptografia, registro de auditoria, log estruturado e monitoramento.

Tabela 4. Correspondência entre os elementos do MVC e as camadas da arquitetura.

Elemento	Camada correspondente	Responsabilidade no DevMatch
View	Apresentação	Renderizar projetos, propostas, contratos, marcos e disputas; capturar as ações do usuário; e apresentar mensagens de validação e de confirmação.
Controller	Interface	Interpretar a requisição, verificar autenticação e permissões, acionar o serviço de aplicação correspondente e devolver a representação adequada.
Model	Aplicação, domínio e persistência	Executar as regras de negócio, manter o estado das entidades, garantir as invariantes e persistir as alterações.

serviços da camada de aplicação, que registram o autor, o instante e o motivo da operação. Dessa forma, nenhuma alteração financeira depende de validações realizadas apenas na interface do usuário.

As principais decisões arquiteturais e suas justificativas estão resumidas na Tabela 5.

4.2 Diagramas de Componentes e de Implantação

4.2.1 Diagrama de Componentes

O diagrama de componentes representa a decomposição do sistema em unidades substituíveis e as interfaces por meio das quais essas unidades se relacionam [**?**]. A Figura 3 apresenta os componentes do DevMatch, agrupados segundo as camadas definidas na Seção 4.1.

A aplicação Web é o componente cliente e consome a interface *IApiRest*, exposta pelo componente de fachada da API. Esse componente concentra a autenticação, o controle de acesso, o roteamento das requisições e a limitação de taxa, delegando a execução aos componentes de negócio. Cada componente de negócio corresponde a um conjunto coeso de casos de uso e fornece uma interface explícita, o que reduz o acoplamento entre eles e permite sua evolução independente.

Os componentes de negócio não acessam o banco de dados diretamente. O acesso ocorre por meio do componente de persistência, que fornece a interface *IRepositorio* e concentra o mapeamento objeto-relacional. De modo análogo, a comunicação com serviços externos é realizada pelos adaptadores da camada de integração, que isolam as particularidades de cada fornecedor. O componente de custódia e pagamentos, por exemplo, requer a interface *IGatewayPagamento*, implementada por um adaptador específico; a substituição do provedor de pagamento exige apenas a troca desse adaptador.

A Tabela 6 descreve os componentes, os casos de uso atendidos e as interfaces fornecidas e requeridas.

4.2.2 Diagrama de Implantação

O diagrama de implantação descreve a distribuição física dos artefatos de software nos nós de execução e os protocolos de comunicação entre eles [**?**]. A Figura 4 apresenta a topologia prevista para o DevMatch.

Tabela 5. Decisões arquiteturais do DevMatch.

Decisão	Justificativa	Consequência
Arquitetura em camadas com dependência unidirecional	Isolar as regras de negócio da tecnologia de interface e de persistência.	Aumenta a testabilidade e permite substituir a interface ou o mecanismo de armazenamento com impacto restrito.
Padrão MVC na interação com o usuário	Separar a exibição dos dados do tratamento das requisições em um sistema com muitos formulários e listagens.	Facilita a reutilização da API por diferentes clientes, incluindo um futuro aplicativo móvel.
API REST sobre HTTPS	Padronizar a comunicação entre o cliente e o servidor e permitir integração com terceiros.	Exige versionamento da API e tratamento explícito de erros.
Custódia implementada como componente próprio, integrado por adaptador	Concentrar as regras de bloqueio e liberação de valores e evitar dependência direta de um fornecedor de pagamento.	Permite trocar o gateway sem alterar as regras de negócio, mas exige conciliação entre os registros internos e os do provedor.
Registro de auditoria transversal e imutável	Atender aos requisitos de rastreabilidade das alterações contratuais e das transações financeiras.	Aumenta o volume de dados armazenados e demanda política de retenção.
Processamento assíncrono de notificações e de recomendações	Evitar que operações demoradas afetem o tempo de resposta das requisições do usuário.	Introduz um intermediário de mensagens e exige tratamento de falhas e de reprocessamento.

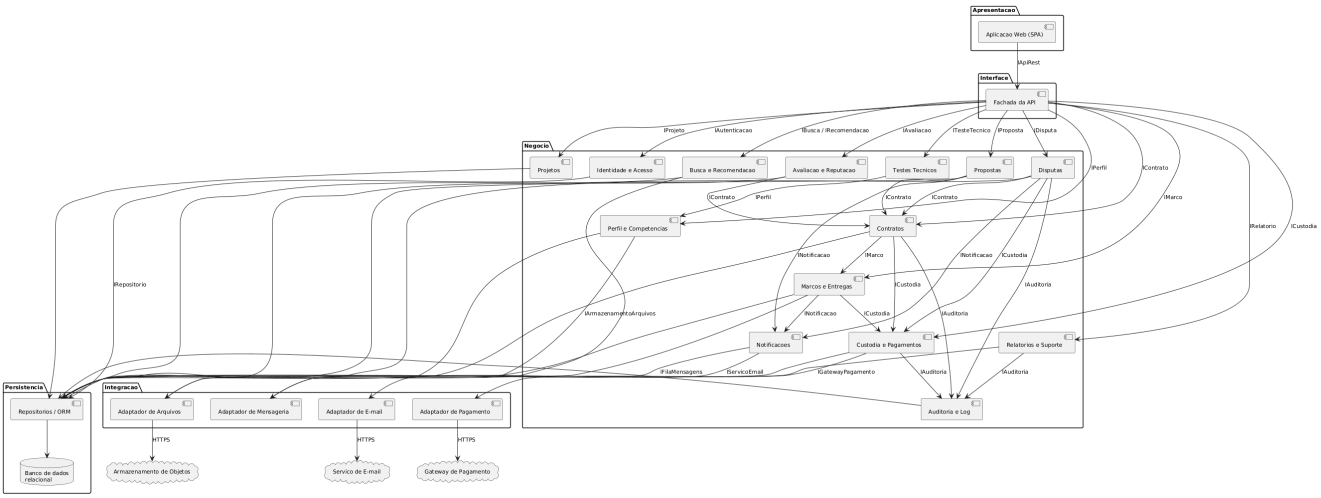


Figura 3. Diagrama de componentes do DevMatch. As interfaces fornecidas são representadas por círculos e as interfaces requeridas por soquetes.

O usuário acessa a plataforma por meio de um navegador Web, que obtém os artefatos estáticos da aplicação a partir de uma rede de distribuição de conteúdo. As requisições dinâmicas são encaminhadas ao balanceador de carga, responsável por encerrar a conexão segura e distribuir as chamadas entre as instâncias do servidor de aplicação. Essas instâncias executam em contêineres e não mantêm estado de sessão, o que permite a escalabilidade horizontal do sistema conforme a demanda.

O servidor de aplicação comunica-se com o servidor de banco de dados relacional, configurado com uma instância primária para escrita e uma réplica para leitura e para a geração de relatórios. O servidor de cache armazena resultados de consultas frequentes, listas de recomendação e dados de sessão de curta duração. Os artefatos enviados pelos freelancers, bem como as evidências anexadas às disputas, são armazenados em um serviço de armazenamento de objetos, e não no sistema de arquivos do servidor de aplicação. As tarefas assíncronas, como o envio de notificações, o recálculo de recomendações e a conciliação financeira, são executadas por nós de processamento próprios, acionados por um intermediário de mensagens.

Os serviços externos são representados como nós distintos. O gateway de pagamento é acessado por meio de chamadas HTTPS e informa a confirmação das operações por *webhooks*, recebidos por um endereço específico da API e validados por assinatura. Toda a comunicação externa utiliza TLS, e as credenciais de acesso aos serviços são mantidas em um cofre de segredos, fora do código-fonte e das imagens de contêiner. A Tabela 7 resume os nós, os artefatos implantados e os protocolos utilizados.

A topologia apresentada atende aos requisitos não funcionais definidos para o sistema. A replicação das instâncias de aplicação e do banco de dados contribui para a disponibilidade; a separação entre requisições interativas e tarefas assíncronas favorece o desempenho; o uso de TLS, do cofre de segredos e do registro de auditoria em nó dedicado atende aos requisitos de segurança e rastreabilidade; e o empacotamento em contêineres facilita a implantação em diferentes provedores de infraestrutura.

4.2.3 Diagrama de Classes

O diagrama de classes do DevMatch representa a estrutura estática das principais entidades do sistema, seus atributos, operações e relacionamentos. A modelagem foi construída a partir dos atores, casos de uso e requisitos identificados na etapa de análise, buscando manter rastreabilidade entre as funcionalidades especificadas e as classes responsáveis por

Tabela 6. Componentes do DevMatch, casos de uso atendidos e interfaces.

Componente	Casos de uso	Responsabilidade	Interfaces
Aplicação Web (SPA)	UC01–UC12	Apresentar as telas do sistema e tratar a interação do usuário.	Requer *IApiRest*.
Fachada da API	UC01–UC12	Autenticar as requisições, aplicar o controle de acesso por papel, rotear as chamadas e padronizar as respostas.	Fornece *IApiRest*; requer as interfaces dos componentes de negócio e *IAutenticacao*.
Identidade e acesso	UC10	Gerenciar contas, credenciais, papéis, verificação de identidade e bloqueio de usuários.	Fornece *IAutenticacao* e *IGestaoUsuarios*; requer *IRepositorio* e *INotificacao*.
Perfil e competências	UC01	Manter perfil profissional, portfólio, certificações, disponibilidade e valor de trabalho.	Fornece *IPerfil*; requer *IRepositorio* e *IArmazenamentoArquivos*.
Testes técnicos	UC01	Aplicar testes, corrigir respostas, registrar pontuação e conceder selos de competência.	Fornece *ITesteTecnico*; requer *IRepositorio* e *IPerfil*.
Projetos	UC04	Publicar, editar, pausar e encerrar projetos e disponibilizá-los para busca.	Fornece *IProjeto*; requer *IRepositorio* e *IBusca*.
Busca e recomendação	UC02	Indexar projetos e perfis, executar consultas com filtros e gerar sugestões com base em tecnologias, disponibilidade, reputação e faixa de preço.	Fornece *IBusca* e *IRecomendacao*; requer *IRepositorio* e *IFilaMensagens*.
Propostas	UC03, UC05	Registrar propostas, manter o histórico de negociação e tratar o aceite ou a recusa.	Fornece *IProposta*; requer *IProjeto*, *IContrato*, *IRepositorio* e *INotificacao*.
Contratos	UC06	Gerar o contrato a partir da proposta aceita, controlar a confirmação das partes e a máquina de estados do contrato.	Fornece *IContrato*; requer *IMarco*, *ICustodia*, *IRepositorio* e *IAuditoria*.
Marcos e entregas	UC07	Definir marcos, registrar entregas e artefatos, tratar aprovações, ajustes e prazos de avaliação.	Fornece *IMarco*; requer *IArmazenamentoArquivos*, *ICustodia*, *IRepositorio* e *INotificacao*.
Custódia e pagamentos	UC08	Registrar depósitos, manter valores bloqueados, liberar repasses, descontar comissões e tratar estornos e reembolsos.	Fornece *ICustodia*; requer *IGatewayPagamento*, *IRepositorio* e *IAuditoria*.
Disputas	UC09	Registrar disputas e evidências, controlar o fluxo de mediação e aplicar a decisão sobre os valores em custódia.	Fornece *IDisputa*; requer *IContrato*, *ICustodia*, *IRepositorio* e *IAuditoria*.
Avaliação e reputação	UC11	Registrar avaliações bidirecionais após o encerramento do contrato e recalcular a reputação.	Fornece *IAvaliacao*; requer *IContrato* e *IRepositorio*.
Relatórios e suporte	UC12	Consultar transações, gerar relatórios financeiros e de comissões e tratar chamados.	Fornece *IRelatorio*; requer *IRepositorio* e *IAuditoria*.
Auditoria e log	UC09, UC10	Registrar de forma imutável as operações sensíveis, os acessos e as alterações contratuais e financeiras.	Fornece *IAuditoria*; requer *IRepositorio*.
Notificações	UC03–UC09	Enviar mensagens por correio eletrônico e notificações internas a partir de eventos do sistema.	Fornece *INotificacao*; requer *IFilaMensagens* e *IServicoEmail*.
Persistência	—	Concentrar o acesso aos dados e o mapeamento objeto-relacional.	Fornece *IRepositorio*; requer o driver do banco de dados.
Adaptadores de integração	—	Encapsular a comunicação com gateway de pagamento, armazenamento de objetos, serviço de correio eletrônico e intermediário de mensagens.	Fornecem *IGatewayPagamento*, *IArmazenamentoArquivos*, *IServicoEmail* e *IFilaMensagens*.

realizá-las.

A classe abstrata `Usuario` concentra informações comuns aos usuários da plataforma. A partir dela são especializadas as classes `Freelancer`, `Cliente`, `Administrador` e `AnalistaFinanceiroSuporte`, correspondentes aos atores identificados na modelagem de requisitos.

O freelancer possui informações profissionais e competências, utilizadas tanto na apresentação de seu perfil quanto no mecanismo de recomendação de projetos. Os projetos são publicados pelos clientes e podem possuir requisitos de competências, além de receber propostas enviadas pelos freelancers.

Quando uma proposta é aceita, ela pode originar um contrato, responsável por formalizar a relação entre cliente e freelancer. O contrato pode ser dividido em marcos, que organizam a execução do projeto em etapas e podem receber entregas realizadas pelo freelancer.

O gerenciamento financeiro é representado pelo pagamento em custódia, associado aos marcos do contrato. O valor permanece retido até que as condições estabelecidas sejam atendidas ou uma eventual disputa seja resolvida.

O sistema também mantém avaliações dos participantes, registros de auditoria, chamados de suporte e relatórios financeiros e operacionais. A Figura 5 apresenta uma visão simplificada das principais classes e relacionamentos do DevMatch. A versão completa do diagrama é apresentada no Apêndice .1.

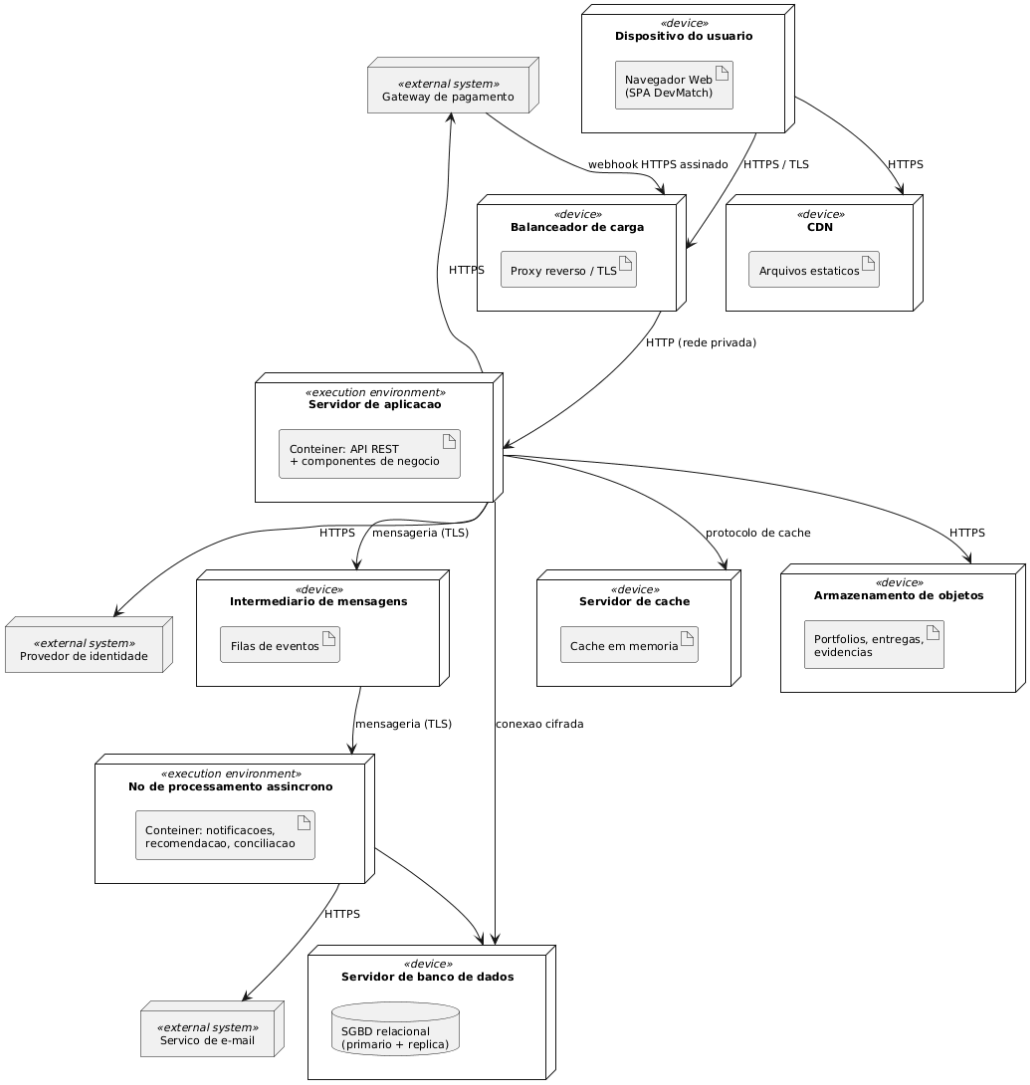


Figura 4. Diagrama de implantação do DevMatch.

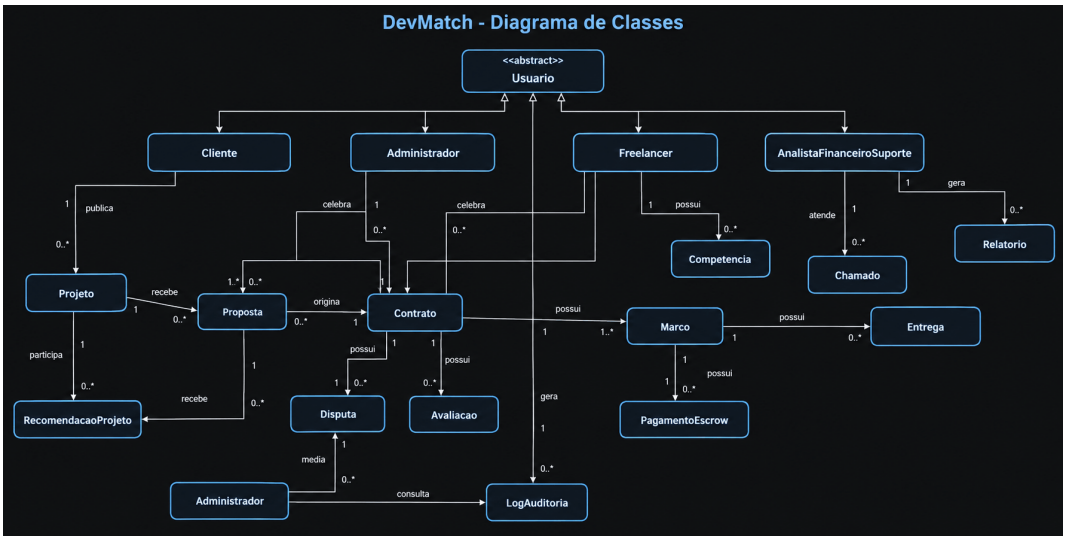


Figura 5. Visão resumida do diagrama de classes do DevMatch.

4.3 Diagramas de Sequência
4.4 Diagramas de Comunicação
4.5 Diagramas de Estados
5 Modelagem de Dados
6 Conclusão

.1 Diagrama de Classes Completo

A Figura 6 apresenta a versão completa do diagrama de classes, relações de herança, associações, composições e tipos enumerados utilizados pela modelagem.

Tabela 7. Nós de implantação, artefatos e protocolos de comunicação.

Nó	Artefatos	Comunicação	Observações
Dispositivo do usuário ⟨⟨device⟩⟩	Navegador Web com a aplicação de página única.	HTTPS/TLS com o balanceador e com a rede de distribuição.	Não armazena dados sensíveis além do token de sessão.
Rede de distribuição de conteúdo ⟨⟨device⟩⟩	Arquivos estáticos da interface.	HTTPS.	Reduz a latência de carregamento e o tráfego no servidor de aplicação.
Balanceador de carga ⟨⟨device⟩⟩	Proxy reverso e terminação TLS.	HTTPS para o exterior e HTTP interno na rede privada.	Distribui as requisições e realiza verificação de integridade das instâncias.
Servidor de aplicação ⟨⟨execution environment⟩⟩	Contêineres com a API REST e os componentes de negócio.	Protocolo do banco de dados, protocolo de cache, HTTPS para serviços externos e mensagens assíncronas.	Instâncias sem estado, replicadas e escaláveis horizontalmente.
Nó de processamento assíncrono ⟨⟨execution environment⟩⟩	Contêineres de tarefas de notificação, recomendação e conciliação.	Mensagens assíncronas e protocolo do banco de dados.	Isola tarefas de longa duração das requisições interativas.
Intermediário de mensagens ⟨⟨device⟩⟩	Filas de eventos do sistema.	Protocolo de mensageria sobre TLS.	Garante a entrega dos eventos e permite reprocessamento.
Servidor de banco de dados ⟨⟨device⟩⟩	Banco de dados relacional, instância primária e réplica de leitura.	Conexão autenticada e cifrada na rede privada.	Armazena dados de usuários, contratos, transações e auditoria; possui rotina de cópia de segurança.
Servidor de cache ⟨⟨device⟩⟩	Cache em memória.	Protocolo de cache na rede privada.	Armazena consultas frequentes e recomendações pré-calculadas.
Armazenamento de objetos ⟨⟨device⟩⟩	Portfólios, artefatos de entrega e evidências de disputa.	HTTPS com acesso por credencial temporária.	Mantém versões dos arquivos e registra os acessos.
Gateway de pagamento ⟨⟨external system⟩⟩	Serviço de processamento de pagamentos e custódia.	HTTPS e *webhooks* assinados.	Responsável pela movimentação financeira efetiva; o sistema mantém o registro conciliado das operações.
Serviços de correio eletrônico e identidade ⟨⟨external system⟩⟩	Envio de mensagens e autenticação federada.	HTTPS.	Utilizados pelos adaptadores da camada de integração.

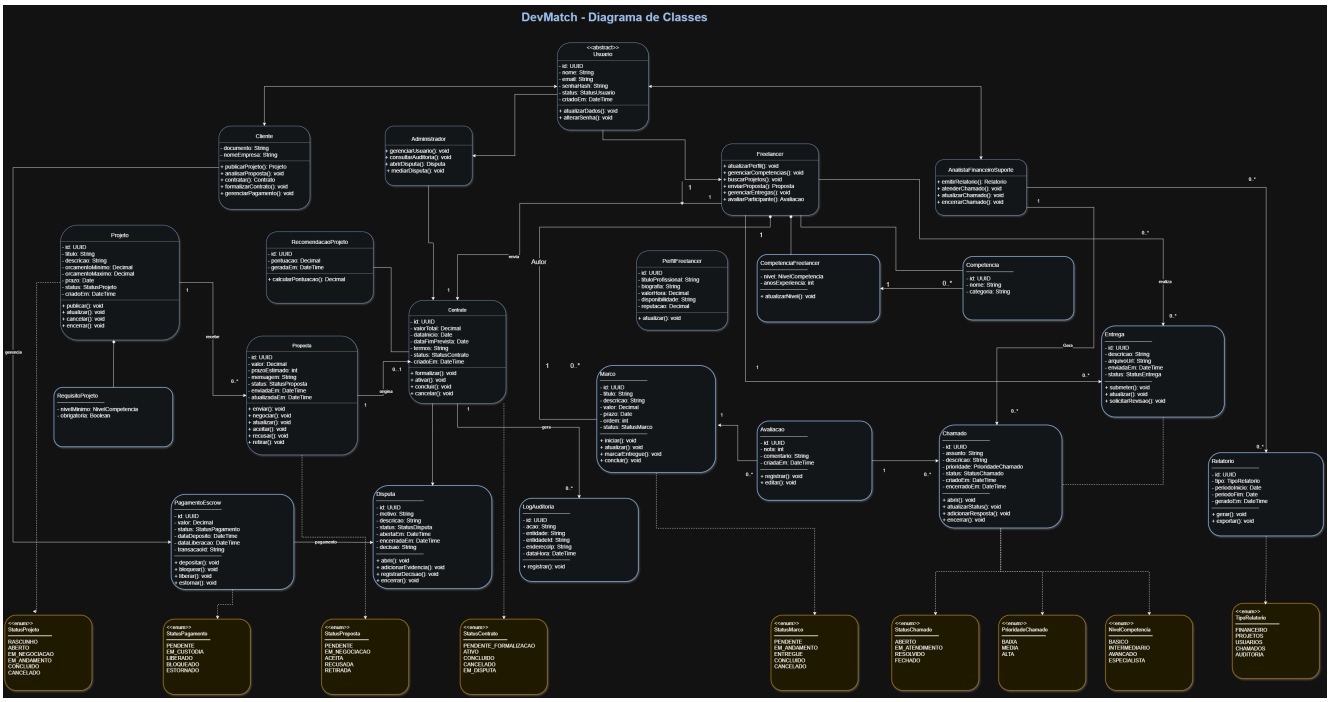


Figura 6. Diagrama de classes completo do DevMatch.

Declarações complementares

Contribuições dos autores

Antônio Henrique Rabelo Andrade contribuiu para o levantamento e a modelagem dos requisitos, a descrição dos atores, a elaboração dos casos de uso e histórias de usuário e a redação da introdução, resumo e abstract.

Todos os autores leram e aprovaram a versão final do manuscrito.

Conflitos de interesse

Os autores declaram que não têm nenhum conflito de interesses.

Disponibilidade de artefatos de pequisa

Os modelos, diagramas e demais materiais produzidos neste trabalho serão disponibilizados mediante solicitação.

Uso de Inteligência Artificial

Foi utilizada uma ferramenta de inteligência artificial generativa, ChatGPT, para auxiliar na organização textual, revisão linguística e formatação inicial de trechos em LaTeX. Os autores revisaram e validaram o conteúdo produzido e permanecem responsáveis por todas as informações apresentadas no trabalho.

Outras informações relevantes

Este trabalho possui finalidade acadêmica e concentra-se na especificação e modelagem de um sistema de software. Não foram realizadas pesquisas ou experimentos envolvendo seres humanos.